The Review–Execution Gap in Agentic Software: Specification, Design Space, and Empirical Characterization

Liang Hanghao[1], Xuan Wentao[1], Chen Qile[1], Peng Peng[1*]

[1]College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, China.

*Corresponding author(s). E-mail(s): hnu16pp@hnu.edu.cn;

Abstract

An agent may regenerate a proposed update after human approval. Even when the final value is unchanged, execution must determine whether the earlier approval covers the new candidate. We compare approval bound to selected context fields, complete candidate content, or the specific reviewed instance. A four-role correction contract links the machine proposal, authorized correction, reviewed candidate and persisted transition, allowing these binding choices to be evaluated under shared execution checks. Nine constructed cases show that context binding can admit semantic changes in omitted fields, whereas content and instance binding reject them. With trusted immutable candidate storage, instance binding also rejects semantically equivalent replacements; none of the three rules alone protects mutable external references. A separate paired agent study isolates the cost of this stricter instance requirement using harness-injected identity-only substitutions at controlled handoffs. Across 32 delivered pairs spanning 16 task definitions, context binding admits every substitution and 30 arms complete their tasks. Instance binding rejects the substitutions. With reviewed candidates available and valid reauthorization requests immediately approved by a deterministic reviewer, all 32 runs recover: 24 reuse the reviewed candidate and eight obtain fresh authorization. Mean additional tool calls per pair are 1.00 in original deployment B and 1.60 in supplementary A, without an established completion advantage. These results distinguish approval coverage from recovery cost, helping developers implement declared approval reuse rules and plan for the agent-side cost of enforcing them.

Keywords: Agentic software, Human review, Runtime enforcement, Authorization, Provenance, Empirical evaluation

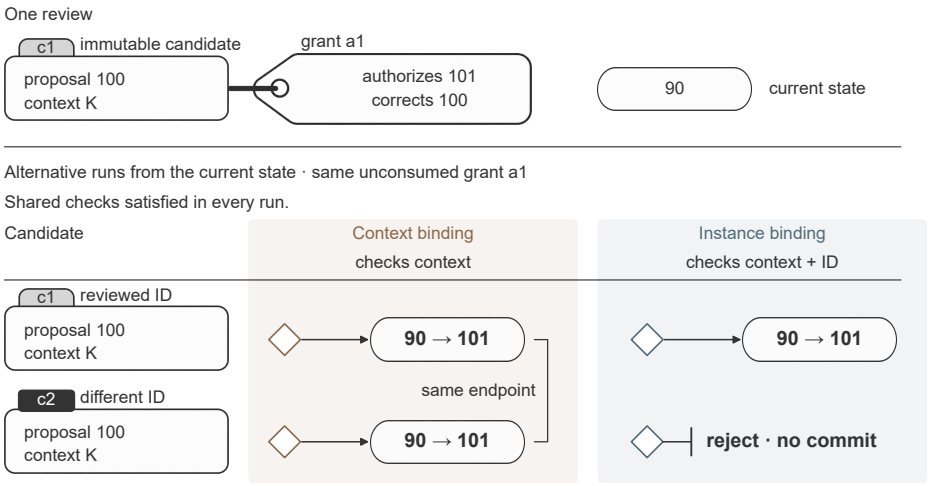


Fig. 1 Equal endpoints do not identify the reviewed instance. A reviewer corrects candidate c1's proposal 100, and grant a1 authorizes 101 for c1. Candidate c2 carries the same proposal and retained context K but a different identifier. Each path represents an alternative execution from the current state 90 with the original grant a1 still unconsumed and the shared checks satisfied. Diamonds mark checks, capsules show the initial state or committed transitions, and the stop bar marks no commit. Context binding admits either candidate and installs 101; instance binding admits only c1. A matching new authorization could license c2, subject to the shared checks. The four roles are the machine proposal (100), the authorized correction (101), the reviewed candidate (c1), and the executed transition ($90 \to 101$). This is a conceptual construction. Object glyphs were AI-generated; see the AI-assisted preparation declaration

1 Introduction

Gerrit's `copyCondition` determines whether a code-review vote carries forward to a new patch set (Gerrit Project 2026a). Agent software faces the same decision when a reviewed update is regenerated or crosses a queued handoff: which replacements may reuse approval? Human-in-the-loop middleware supports approval, argument edits, rejection and resumption (LangChain 2026); the application must still relate its persisted grant to the candidate eventually written.

Consider persisted candidates c1 and c2 with the same proposal 100, evidence, producer and expected predecessor, but different identifiers. A reviewer inspects c1 and authorizes 101; a later handoff supplies c2 under that grant and installs 101. Value, predecessor and retained-context checks pass, yet execution uses a different candidate. This is the **review–execution gap**. A policy permitting equivalent candidates can accept c2; one attaching responsibility to the reviewed artifact must preserve c1's identity. Endpoint correctness does not imply continuity of the reviewed candidate.

We characterize this choice with a four-role contract linking the machine proposal, authorized correction, reviewed candidate and persisted transition. Context binding accepts candidates with equivalent retained review information; instance binding additionally requires the reviewed instance. Both share authorized-value equality, predecessor freshness, an atomic complete successor and field-source preservation.

Table 1 Contributions and their principal evidence.

Contribution	Principal evidence	Section
1. Compare binding choices	Nine constructed comparator cases; conditional inclusion	5.3
2. Characterize continuation	Frozen paired suffixes; recovery event order	6.3
3. Specify the correction relation	Fixed-registry witness; local observation model	3.3
4. Realize and validate obligations	Store checks; external ToolGate adaptation	5

Paired execution from the same review checkpoint isolates the instance requirement from the agent's response.

Our design space organizes which review object to retain, which relationships to check before writing and which continuation a failed check should expose. Store implementations and independent persistence checks connect these choices to durable effects; an external-framework study maps the obligations to existing extension interfaces.

Two comparisons separate approval coverage from continuation cost. Nine constructed cases distinguish omitted-field changes, content-equivalent replacements and mutable external dependencies. The paired agent study holds retained content fixed ($K = R$) and measures recovery after identity-only rejection, rather than the benefit of preventing semantic change. Native LangChain and LangGraph probes locate the boundary between interrupt approval and application-level binding of a persisted business grant.

For builders of agentic software, the paper makes four contributions (Table 1):

1. **A comparison of approval-binding design choices.** We organize value, context, content and instance binding under shared checks, nested compared fields and immutable records, and expose their boundaries in nine constructed cases.
2. **Measured agent continuation under an instance requirement.** Paired execution isolates instance equality; event-order analysis separates recovery commits, subsequent confirmation and complete paired-suffix cost.
3. **An explicit contract for corrected execution.** Value, freshness, successor and source obligations connect the four roles. A constructive result shows why equal predecessors and final values do not determine review continuity.
4. **Executable enforcement and persistence-based validation.** Store implementations, controlled cases, bounded Alloy checks and an independent auditor validate durable effects. A ToolGate adaptation combines normal callbacks with an author-written commit bridge on public reference workflows.

Table 2 Binding, mismatch handling and evidence in the cited sources. The two attack analyses describe failure paths; their check points are observation points.

Mechanism	Binding object	Mismatch handling	Check point	Evaluation form
LangChain HITL (LangChain 2026)	Interrupted tool call	Reviewer may edit/reject, then resume	Before tool execution	Interface documentation
ReSA (Zhu et al. 2026)	Grant scope; consumed versions	Derive repair obligations	Reconstruction to local sink	Authored graphs; installed-control checks
Guarded Commits (Bindschaedler et al. 2026)	Artifact, policy; edit hashes	Fresh authorization or review	Commit	Public-trace replay; synthetic DAGs
Cordon (Chen et al. 2026)	Task, lineage, staged effects	Abort, rollback, compensation	Commit or effect release	Agent workflows; scripted rollback/resume
Continuity Kernel (He and Yu 2026)	Proposal ID; sealed successor	Reject, quarantine or defer; new-ID retry	Atomic activation	Bounded model exploration
Commit-time authorization (Santos-Grueiro 2026)	Witness, target, eligibility	Abort or refresh/rebind	Durable-effect boundary	Agent perturbations; replay; limited enforcement
ToolGate (Liu et al. 2026)	Symbolic state; tool contracts	Filter call; discard invalid result	Tool invocation; state update	ToolBench; MCP-Universe
Gerrit / GitLab (Gerrit Project 2026a,b; GitLab 2026)	Patch set / merge request	Copy or reset votes by policy	New revision; submit/merge	Product documentation
OpenTofu (OpenTofu 2026)	Saved plan decisions	Plan mismatch: not assessed in cited guide	Plan application	Command documentation
Loopjacking (Kumar 2026)	Analyzed: call identity and arguments	Substitution attack; deny-update control	Review, update, resume	Released Agno; configured LangGraph tests
Approval Laundering (Wang 2026)	Analyzed: grant, dispatch, effects	Policy divergence; token-replay check	Dispatch; effect sinks	Claude Code probes under research policies
This work	Compared: context, content and instance	Reuse reviewed instance or reauthorize	Atomic successor write	Formal/store checks; paired agent continuation

2 Related Work

2.1 What existing mechanisms bind

Existing mechanisms bind authority to different objects: an interrupted call, patch set, saved plan, sealed proposal, commit-time witness or symbolic-state predicate (Table 2). Code review makes applicability configurable through vote copying, approval reset and submit requirements (Gerrit Project 2026a; GitLab 2026). We study that choice for corrected agent state and its consequences for continuation.

ReSA derives approval-scope repair obligations from consumed versions, trusted preservation and atomic validation (Zhu et al. 2026). Guarded Commits checks path coverage and pinned policy, binds EDIT decisions to pre-edit, delta and post-edit hashes, and creates a fresh record for valid decision reuse (Bindschaedler et al. 2026). Our correction relation organizes the machine proposal, authorized correction, reviewed candidate and installed transition with field-source preservation. Holding context,

authorized values, freshness and persistence checks fixed, we compare whether approval covers a content-equivalent candidate or only the reviewed instance, then measure agent recovery and cost.

Continuity Kernel seals proposal identity and exact candidates through atomic activation. Commit-time authorization distinguishes endpoint success from valid authority at durability and includes recovery comparisons (Santos-Grueiro 2026). Cordon studies workflow execution and approval costs. These mechanisms supply foundations for our controlled binding comparison; ToolGate's application pre/postconditions provide an external implementation target (Section 5.2).

Loopjacking exposes changed tool arguments executing under a review of different arguments in Agno and a LangGraph Agent Server composition (Kumar 2026). Approval Laundering examines divergence between approved invocations and dispatched actions or downstream effects in Claude Code (Wang 2026). Our paired study examines the complementary identity-only case: correction and retained context remain equal, and the selected binding rule determines whether the agent must recover.

2.2 Foundations and the correction relation

Transactions and concurrency control supply atomicity and predecessor validation (Gray 1981; Kung and Robinson 1981). Provenance covers derivations, edits and copies (Buneman et al. 2001; Cheney et al. 2009; Buneman et al. 2006), transaction reenactment (Arab et al. 2018) and entity/activity/agent descriptions (World Wide Web Consortium 2013). Interactive repair separates machine suggestions from human judgments (Yakout et al. 2011; He et al. 2016). Our contract joins these concerns in the installed successor's authorized correction, reviewed-candidate link and field sources.

Integrity and capability systems connect principals, permitted operations and protected objects (Clark and Wilson 1987; Hardy 1988; Miller 2006). Runtime verification and monitoring address execution conformance and enforceable policies (Leucker and Schallhart 2009; Schneider 2000); information-flow control constrains propagation (Sabelfeld and Myers 2003), and accountability supports reconstructible, attributable execution (Haeberlen et al. 2007).

Table 3 maps the obligations defined below to their existing foundations and their role in this study. This composition separates three diagnostic questions: whether the old grant covers the submitted candidate, whether the transition uses the reviewed instance, and whether the successor preserves the required field sources. The agent study then observes continuation after rejection under the instance rule, separating recovery commits from later confirmation. These connected questions and observations define our increment over the reused mechanisms; the table does not claim that the cited systems cannot express these relations.

3 A Contract for Corrected Execution

3.1 Four roles

Figure 1 illustrates the four roles: proposal 100, authorized correction 101, reviewed candidate c1 and the transition from 90 to 101.

Table 3 Existing foundations and their role in the corrected-execution comparison. P1–P6 are defined in Section 3.2.

Obligation	Existing basis	Role in this study
P1	Continuity Kernel binds proposal and sealed candidate; ReSA tracks the consumed object (He and Yu 2026; Zhu et al. 2026).	Vary reviewed-instance equality while holding the correction and shared checks fixed; observe continuation.
P2	ReSA requires preserved or atomically validated consumed versions (Zhu et al. 2026).	Compare retained context and declared content; separate external referent checks.
P3	Guarded Commits binds authority and pre-edit, delta and post-edit hashes (Bindschaedler et al. 2026).	Retain the machine proposal separately from the authorized correction and committed value.
P4	Optimistic concurrency control and Continuity Kernel validate predecessors (Kung and Robinson 1981; He and Yu 2026).	Keep freshness shared across policies, separate from instance mismatch.
P5	Transactions and Continuity Kernel install an atomic accepted unit (Gray 1981; He and Yu 2026).	Install the complete corrected successor, sources, transitions and grant consumption together.
P6	Provenance and curated-database copying retain derivation and source relations (Buneman et al. 2001, 2006).	Link changed fields to authorized transitions and copy unchanged fields' exact prior sources; check these relations in persistence.

A persisted candidate is

$$c = \langle \mathrm{id}, r, f, e, v_{\mathrm{exp}}, x_c, p \rangle$$

where id names a content-immutable persisted candidate record, (r, f) the record and field, e the evidence and locator, v_{exp} the expected predecessor, x_c the proposal, and p the producer. An authorization $a = \langle c_r, x_a, q \rangle$ records the reviewed candidate, explicitly authorized value and authorized principal. Execution submits candidate c_s and value x, producing a persistent transition. These elements define the four roles: the **machine proposal**, the **authorized correction**, the **reviewed candidate** and the **executed transition**. The proposal remains in the record when correction makes $x_c \neq_{\mathrm{json}} x_a$; the successor must contain x_a.

Retained context $K(c)$ comprises record and field, evidence identity and locator, expected predecessor, proposal and producer. Candidates are *context-equivalent* when they agree on K, and *content-equivalent* when they agree on declared fields $R \supseteq K$, under the specified comparator. *Same-semantics* means the same approved effect under the scenario oracle. An *identity-only substitution* preserves the declared tuple's non-identifier components while changing its identifier. Instance-creation and outer envelope metadata lie outside this tuple. Value comparisons use canonical JSON equality.

3.2 Admission and persistence

Define context and identity agreement by

$$K_{\text{eq}} \Leftrightarrow K(c_s) = K(c_r), I_{\text{eq}} \Leftrightarrow \text{id}(c_s) = \text{id}(c_r)$$

Let G collect the shared checks: valid authority and grant, context/evidence integrity, authorized-value equality, current predecessor, complete successor, source relations and transaction constraints. The integrity component of G concerns whether the submitted candidate is internally consistent with its evidence; it is separate from K_{eq}, which compares the submitted and reviewed contexts. The two policies are

$$P_{\text{context}} : \text{Admit} = G \wedge K_{\text{eq}}$$
$$P_{\text{bound}} : \text{Admit} = G \wedge K_{\text{eq}} \wedge I_{\text{eq}}$$

Both policies retain the reviewed identifier in the authorization; they differ only in the execution rule. Context binding treats the retained information as the approval object; instance binding attaches responsibility to the reviewed artifact. A same-key retry retrieves the existing result, whereas re-preview can materialize a new candidate requiring a matching grant.

The execution contract combines six obligations: **P1**, instance binding under the bound policy; **P2**, context and evidence integrity; **P3**, principal authority and the authorized value; **P4**, predecessor freshness; **P5**, one complete atomic successor; and **P6**, field-source relations. Changed fields point to their authorized transitions; unchanged fields retain their exact prior sources. The successor, transitions, sources and grant consumption commit atomically. Direct-write exclusion (P0) places all authoritative mutation behind this boundary.

Trusted candidate storage is a separate prerequisite: a candidate identifier must resolve to one content-immutable record, and the executor must consume that record. Candidate creation does not confer permission to overwrite an existing candidate or its review linkage. P0 excludes bypass writes to authoritative state; it does not by itself establish this storage prerequisite. External referents require the separate consumption check in Section 5.3.

3.3 What endpoint correctness cannot establish

The exact policy difference is the request-state set

$$S = \{r : G(r) \wedge K_{\text{eq}}(r) \wedge \neg I_{\text{eq}}(r)\}$$

so bound admission refines context admission. The following witness illustrates what endpoint checks leave unresolved.

Endpoint indistinguishability. *Equal predecessor states and equal final values can carry different review continuity.* Take independent copies of the same pre-admission state and unconsumed grant. In one copy, context binding admits (c_s, x_a); in the other, instance binding admits (c_r, x_a). When the candidates differ only in identity, both successors have the same field-wise values and copied sources, but only the second transition uses the reviewed instance. The archived fixed-registry witness isolates this difference while holding the other observations fixed. Because the two executions also

agree on retained context, a check that reads only final values, predecessor freshness and retained context assigns both the same verdict; continuity must be observed through the candidate–authorization binding itself.

With authorization parameters fixed, five information classes suffice for the declared local classification: candidate–authorization binding, role-labeled values, predecessor discriminators, successor structure and field sources. Online Resource 2, A9 defines the classification and gives witness pairs establishing irredundancy; Online Resource 1, S2 supplies executable checks.

Gate soundness is conditional on direct-write exclusion, a trusted gate and adapter, authority checked at commit entry, consistent equality, transaction-scoped reload and predecessor validation, and atomic installation of the checked relations. Under these premises, a bound commit satisfies the locally checked obligations (P1–P5 and the declared local source checks). Full trace soundness (P6) additionally requires a trace-complete predecessor, immutable valid references, validated new chain links and uniqueness of the successor's transitions; this extension is supported by bounded preservation checks and audits rather than by a general proof. Conversely, a nonempty change that satisfies the remaining contract conditions is accepted when the gate observes the authoritative state and its grant and key remain usable; the detailed acceptance conditions are in A9.

3.4 Continuation after rejection

For an identity-only mismatch, the agent can reuse the reviewed candidate under its still-valid grant or obtain/use a matching authorization for another candidate. Changed evidence or a stale predecessor requires a new candidate reflecting current state and matching authorization. A consumed grant with a prior commit instead calls for receipt retrieval or state confirmation. Each new admission must satisfy the remaining checks.

A rejection thus identifies the failed relationship, and the continuation shows whether an agent can restore it. Section 6 measures those subsequent actions and their cost.

4 A Design Space for Binding Review to Execution

The contract makes the approval object an explicit engineering choice. If responsibility attaches to a reviewed artifact, instance binding preserves that relationship even when final values agree. If the application permits interchangeable candidates, context binding expresses that rule. Under immutable records and shared checks, stronger binding narrows admission; reauthorization instead changes the continuation branch. API-specific comparison semantics remain essential (Lodderstedt et al. 2023).

Three axes organize this choice (Figure 2). The first is the binding object: no gate, value, context, content or instance. Under trusted lookup of content-immutable records, identity equality implies content equality and hence context equality. Writing $P_{\text{value}} = G$, and defining content binding on fields $R \supseteq K$, gives $P_{\text{bound}} \subseteq P_{\text{content}} \subseteq P_{\text{context}} \subseteq P_{\text{value}}$ under shared checks. Thus K_{eq} is redundant under instance binding. The adapter must preserve the resolved immutable record to use: the trusted-preservation case of ReSA (Zhu et al. 2026), with external documents checked separately at consumption.

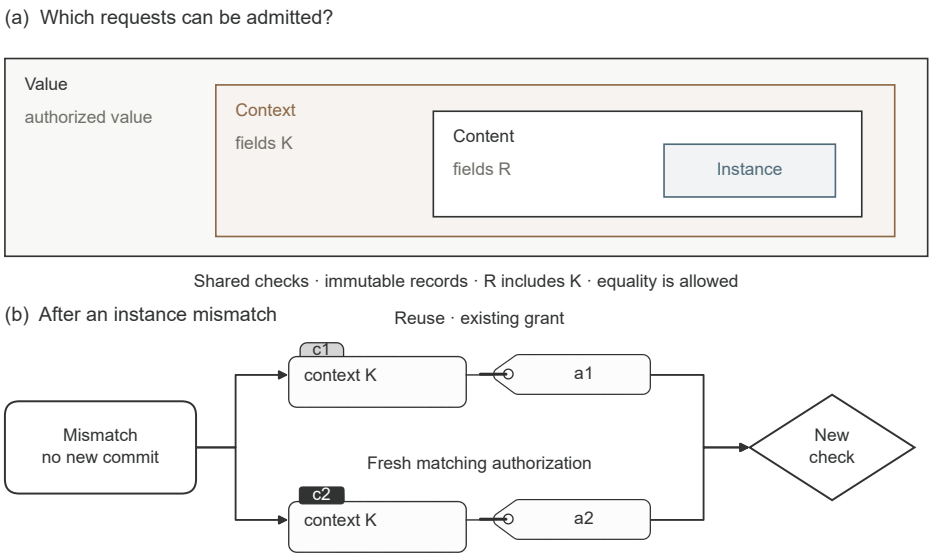


Fig. 2 Binding and recovery are separate design choices. Under shared checks and content-immutable candidate records, if the compared content fields R include the retained context fields K, the admissible request sets satisfy $A_{\text{instance}} \subseteq A_{\text{content}} \subseteq A_{\text{context}} \subseteq A_{\text{value}}$. These inclusions need not be strict; context and content binding coincide when $K = R$. Following an instance-mismatch rejection, the system may offer reuse of the reviewed candidate under the existing authorization or obtain matching authorization for another candidate. Reauthorization is a recovery branch, not another binding level. Each continuation must pass a new admission check; the schematic does not guarantee that an available route will be taken. Object glyphs were AI-generated; see the AI-assisted preparation declaration

The contract and online G handoff use $K = R$ over the declared non-identifier fields, excluding outer host metadata; the levels separate when retained context omits a decision-relevant field (Section 5.3).

The second axis is mismatch handling: adoption with a substitution record, unexplained denial, actionable rejection or routing to reauthorization. The tested context policy adopts the substitute and records its identifier. Reauthorization is a continuation branch, not another binding level: it retains the instance decision at admission but provides a matching new grant. The third axis is the check's anchor and timing: request, server-held review record or persisted constraint, checked at review, handoff, commit or grant consumption.

Online Resource 2, A10 maps responsibility rules, compared fields, trusted persistence and continuation to implementation choices. Section 5.3 compares three binding rules deterministically; Section 6 tests agents under context binding with recorded substitution and instance binding with a continuation channel. The remaining combinations are design options.

Two responsibility choices.

Consider a design example: a release reviewer corrects a configuration in an immutable change record, and the owner requires the installed transition to name that record.

Rebuilding identical content preserves the effect but loses this link. Recording the substitution does not authorize approval transfer; reuse under the valid grant or authorization for the replacement preserves the rule. Alternatively, the owner may permit carryover whenever the complete configuration, evidence and predecessor agree. Content binding with a recorded substitution then suffices. Gerrit's configurable vote copying illustrates such explicit project policy (Gerrit Project 2026a). The owner's responsibility rule therefore determines the binding and recovery choices.

5 Realization and External Comparison

5.1 Execution gate and independent checks

The contract runs in a relational store and an event journal. The relational adapter reloads state, evidence and authorization within the transaction, validates the principal and unused identities, compares-and-swaps the predecessor, and installs the complete successor, transitions and exact sources. The journal records the same relations under context and exact configurations differing only in instance equality. An independent database oracle reads persisted pre/post relations; a source verifier follows field links through transitions, authorizations, candidates, evidence and predecessors.

The controlled suite runs 165 cases per mechanism. The context journal accepts 60 and rejects 105; the exact journal and relational reference each accept 45 and reject 120. The 15 differences are same-context substitutions. The two exact implementations also agree on provenance-query answers. Request/display probes and the case mapping appear in Online Resource 2, A6.

Bounded formal checks and a historical audit support the same relations. Alloy/Kodkod checks (Jackson 2002; Torlak and Jackson 2007) cover admissions, conjunct removal, source attacks and trace preservation: 72 command outcomes match the manifests, with no assertion counterexamples in the encoded finite scopes. The independent mapping has nine intended satisfiable projections and twenty unsatisfiable mutants. A historical audit reconstructs 84 machine-origin correction chains from 1,260 hosted databases, checking 252 evidence associations, 168 record versions and 168 provenance traces. It establishes that the four roles can be recovered from the retained persistence records; unchanged-field copying is exercised in the controlled multi-field cases (Online Resource 1; Online Resource 2, A5–A6).

5.2 ToolGate portability case study

ToolGate (Liu et al. 2026) supplies symbolic state and application pre/postconditions. Its normal callbacks express the binding, value, predecessor and source obligations; an author-written SQLite bridge supplies the record schema and durable commit boundary. This tests implementability through extension interfaces, not native default admission.

Execution has four stages: callbacks check the request; the native tool drafts changes in a cloned world; postconditions validate the draft; the bridge installs the successor and its relations in SQLite. Persisted state is then projected back into the public world for the reference suffix (Online Resource 2, A8).

Table 4 Configured ToolGate (T) versus the contract backend (C). Development controls and public workflow routes are separate units; decisions and recovery are scripted.

Check	T	C
Development controls satisfied	8/8	8/8
Public: initial authorized business commits	12/12	12/12
Public: invalid requests rejected without a business write	12/12	12/12
Public: fixed legal recovery commits	12/12	12/12
Public: complete reference-world matches	24/24	24/24
Public: frozen strict PASS/FAIL	23/1	24/0

Six public workflows crossed with four controlled scenarios give 24 routes per implementation. The retained strict failure is two initialization log lines violating an empty-output rule. Post-hoc inspection confirms that route's business relations and final reference world; the frozen FAIL is unchanged. Reference-world agreement is not an official benchmark success rate or evidence of autonomous recovery.

Both implementations pass eight development controls. Six public Retail workflows from the pinned tau2-bench repository (Barres et al. 2026), crossed with four scenarios, yield 24 routes per implementation (Table 4). Both satisfy the shared business checks and scripted recovery. ToolGate's one strict failure is two initialization log lines violating the empty-output rule; the frozen failure is retained separately from business-state scores. Scripted decisions establish implementability on these workflows (Online Resource 2, A8).

5.3 Binding scope and native defaults

Two deterministic experiments characterize resume routing and the fields each binding comparator checks. Neither calls a model. Online Resource 2, A11 identifies the protocol, saved runs and reproduction commands.

Native human-in-the-loop defaults. We test unmodified LangChain middleware (LangChain 2026) and the LangGraph interrupt primitive (langchain 1.2.10, langgraph 1.0.8) with an author-written graph, executor, recorder and primitive payload schema. Sixteen deterministic conditions record 18 SQLite effects. Two-turn conditions retain one graph, checkpointer and thread. In both mechanisms, a second turn creates a new interrupt without writing. Resuming with the earlier identifier leaves approval pending and adds no write; the current identifier permits one. Repeating affirmative input at the new interrupt also writes once, constituting fresh approval rather than reuse. An explicit edit from 100 to 999 persists 999 while earlier checkpoints retain the original proposal and review payload; malformed middleware decisions raise errors without writing. Thus approval addresses the pending interrupt, and no earlier approval was consumed by a later request in these routes. Binding a persisted business grant remains an application decision. The probe covers successive interrupts and edits, not same-identifier pending-call replacement or Loopjacking's Agent Server configurations (Kumar 2026).

Table 5 Binding-gate decisions over nine constructed cases. A: admit; R: reject. Each decision is identical under unchecked and verified execution (72 routes); entries do not report writes. All comparators share the authorized-value check.

Case	Context	Content	Instance	Raw content (diagnostic)
C1: Identical new instance	A	A	R	A
C2: Effective-date change[a]	A	R	R	R
C2: Cost-center change[a]	A	R	R	R
C2: Date change + render noise[a]	A	R	R	R
C3: Changed external document[b]	A	A	A	A
C4: Number/string representation	A	A	R	R
C4: Display-only note	A	A	R	A
C4: Semantic note	R	R	R	R
C3: Unchanged-document control[b]	A	A	A	A

[a]These three changes affect fields omitted from the context key. Context admission produces semantic-violating writes under both executors; content and instance binding reject them.
[b]Every binding admits the changed external document. The common verified executor blocks its write; unchecked execution writes 999.00. The unchanged-document control writes 101.00 in both modes.
Raw content omits money normalization and is diagnostic. A note is compared when the case declares it semantic.

Binding scope. Nine constructed cases cross three binding comparators and a diagnostic variant with two execution modes, giving 72 routes (54 principal, 18 diagnostic). All arms apply the same authorized-value guard and differ only in the compared fields (Table 5). In one case the reviewed document changes after review while the candidate stays identical, and a matched control leaves it unchanged. An unchecked executor reads the live document; a verifying executor checks the reviewed digest and consumes the same captured bytes, stopping on mismatch. An independent scorer reads only the persisted effects and compares them with the approved effects saved before execution; it detects deliberate deletion, addition and alteration in copies of each sink.

This experiment uses an extended candidate schema: K contains the target record, operation, unit, source-document digest and proposal; R adds effective date and cost center. A note enters both projections only when declared semantic. Monetary values are normalized to two decimal places. These comparators and the common executor test binding scope, not the full shared gate G. In contrast, the contract's declared tuple and the online G handoff use $K = R$ with canonical JSON equality.

Context binding admits three changes to the omitted effective-date or cost-center fields; checking the external document's digest does not cover those fields. Content binding rejects the changes through field equality. Instance binding rejects them because each immutable-record change creates a new identifier, avoiding field enumeration in the reuse key. It also rejects three same-semantics re-materializations admitted by content binding: an identical new instance, representation change and display-only note. Section 6 measures recovery from identity-only substitutions. The changed-document case passes all four bindings because the candidate is unchanged: the unchecked executor writes the changed amount, the verifying executor writes nothing, and both write the reviewed amount in the unchanged control. Referent protection comes from the executor's consumption check.

6 Paired Agent Execution

6.1 Design and measurement

Both policies run from the same review checkpoint, so the instance conjunct is the only difference between paired arms. The original study freezes 128 tasks across two provider configurations and two policies: 512 planned arms, one realization per cell. The six families are legal continuation (L, 32), refreshed preview (N, 32), queued handoff (G, 16), changed evidence (E, 16), stale predecessor (V, 16) and receipt unavailability (R, 16). In both collections, all families share an initial instruction not to treat a preview candidate as automatically authorized; N additionally states that a preview does not itself authorize a commit. A deterministic reviewer creates a shared authorization; any candidate that passes validity checking is granted immediately, and a fresh authorization supersedes earlier grants. The conversation and persistence checkpoint are then copied into isolated context and bound arms with balanced policy order (Figure 3).

At the queued handoff (G), the harness submits a standardized context-equivalent substitute, after which actual gate feedback enters the next agent decision. The refreshed-preview family (N) leaves management of a required refreshed preview to the agent. Receipt unavailability (R) makes the first receipt query unavailable after a successful commit response has already been delivered; it covers only this confirmed-commit path, and unknown-outcome recovery is a separate deterministic probe (Online Resource 2, A3).

The design separates exposure from behavior. At G, the harness creates and submits the substitute; context admission and the resulting $I = 0$ follow from the policy and oracle definitions, so the 17 and 15 substitution counts are exposure counts rather than agent actions. The behavioral observations begin with the delivered outcome: whether context arms complete, which legal route bound arms choose, their action sequence through the first persistent recovery commit, and any subsequent confirmation calls.

The online host is a separate reduced implementation shared by A and B, not the relational gate of Section 5. Tasks commit one field and record its source as the successor transition identifier, exercising reduced P5/P6. Controlled implementations test multi-field successors, deep source chains and the full evidence graph. With deterministic grants for valid candidates, reauthorization measures agent routing rather than human review burden.

Configuration A requests `gpt-5.6-terra` through Codex CLI 0.160.1 with low reasoning; no returned model identifier is exposed. B requests and receives `deepseek-v4-flash` through an Anthropic-compatible Messages API at temperature zero. Both use the same execution system. Each suffix permits 20 decisions and 24 tool calls, with a 90-second request timeout and a 720-second suffix limit. One format-repair allowance applies across the full prefix-plus-suffix history. B has a 4,096-token response cap; A's upstream cap is unavailable. Frozen inputs and complete settings are in Online Resource 1.

Collection provenance. A provider quota prevented 117 original A pairs from receiving any model response before review. Those previously unexposed pairs form a separately frozen supplementary A collection of 234 arms, retaining task definitions,

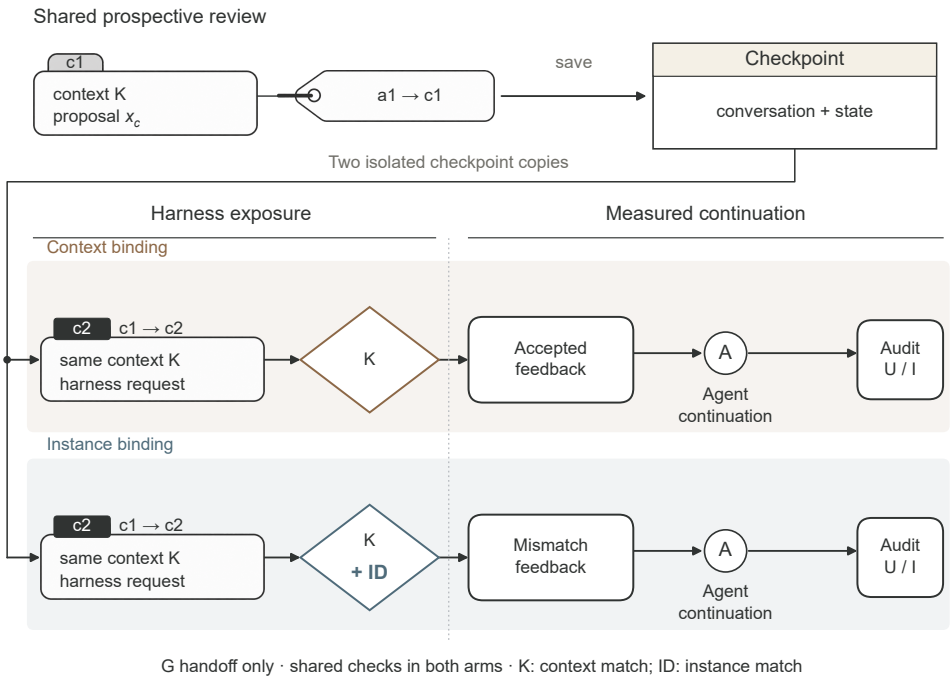


Fig. 3 At standardized G handoffs, paired gates differ only in instance equality. The reviewed candidate c1 and its authorization a1 are saved with the conversation and state, then copied into two isolated, directly labelled policy arms. The harness submits c2 with the same context and a different identifier. K denotes K_{eq}; the instance gate adds ID, denoting I_{eq}, while all other checks are shared. Both admission and rejection produce actual feedback for the next sampled agent decision (A), followed by an independent U/I audit. A rejected request makes no commit but leaves the continuation active; subsequent requests must still pass their arm's gate. Measured G outcomes cover 32 delivered pairs from 16 task definitions. N leaves preview management to the agent and is not depicted. Object glyphs were AI-generated; see the AI-assisted preparation declaration

prompts, tools, policy ordering, requested configuration and scoring. Original outcomes remain unchanged; the later, selected collection has its own denominator and exploratory analysis.

The independent oracle scores utility U from authoritative goal attainment, required workflow and accurate final reporting. Integrity I is 1 with sufficient coverage and no contract violation. A proved violation yields 0, and incomplete necessary evidence yields unknown unless a violation is already proved. With no authoritative transition, integrity can hold vacuously and carries no evidence about contract conformance. The instance-continuity oracle applies to both policies, so a context-admissible substitution can score $I = 0$: $I = 0$ records non-conformance to the declared instance rule, not a safety failure under the context rule that the arm declares. Admissibility under a policy and conformity to the independent oracle are different judgments.

For aggregate completion differences, each task contributes the mean of its configuration-specific paired differences (bound minus context). We bootstrap 128 task clusters within family, retaining both policies and configurations (10,000 iterations,

Table 6 Completion and instance substitution by collection. Context (Ctx) and instance (Inst; archived as bound) counts are separate; completion denominators remain the planned pairs.

Collection	Planned pairs	Recorded checkpoints	Completion		Substitutions		I unknown	
			Ctx	Inst	Ctx	Inst	Ctx	Inst
Original A	128	11	9	9	1	0	0	0
Original B	128	127[a]	117	119	16	0	1	1
Supplementary A	117	117	114	116	15	0	0	0

Recorded checkpoints count pairs whose frozen ledger explicitly records reaching the shared review checkpoint. Original A includes 117 pairs stopped by quota before review; supplementary A evaluates those pairs later. [a]Original B also has one partially recorded N pair without that ledger flag: its context arm retains a partial after-state and its bound arm was not started. It is not reclassified as failing to reach review. These are the two unknown arms. I denotes the independent instance-rule conformance score; arms without an authoritative transition can satisfy it vacuously.

seed 20261007). We apply the same procedure separately to the 117 supplementary pairs. Recovery summaries condition on feedback in a saved model input followed by a received response. Recovery calls run from that feedback through the first successful commit that adds a persistent transition; subsequent calls are counted separately. Complete-suffix cost remains the bound-minus-context difference over both full continuations.

6.2 Task success hides substitution

Substitutions that context binding admits mostly complete their tasks. All proved substitutions occur at G under context binding: 17 in the original collection and 15 in supplementary A. Thirty of these 32 arms still complete their task, so an audit of endpoint utility alone would accept them; the continuity oracle separates them. The 32 episodes come from 16 G task definitions, each observed under deployments B and A. The corresponding bound arms reject the substituted request and all finish with $U = 1, I = 1$. Table 6 retains complete cohort denominators, including quota failures and unknown integrity; family-level joint outcomes are in Online Resource 1, S11–S12.

Observed completion rates are close under the two policies. Across the original collection, completion is 126/256 versus 128/256: +0.78 percentage points, with a 95% task-cluster interval of [−1.95, 3.52]. This pooled estimate is dominated by deployment B: original A contributes 11 reviewed pairs and no completion discordance, so the pooled difference is approximately half of the B-specific difference (+1.56 points [−3.91, 7.03] in original B). The supplementary difference is +1.71 points [0.00, 4.27]. Neither comparison establishes a completion advantage (Section 7.2). Original B has 18 capped invalid-output terminations (10 context, 8 bound); this imbalance constrains interpretation of the original-B completion difference.

A later 24-task output-budget check gives context/bound completion of 23/24 versus 22/24 at 4,096 tokens, and 24/24 for both at 393,216 tokens. Integrity remains

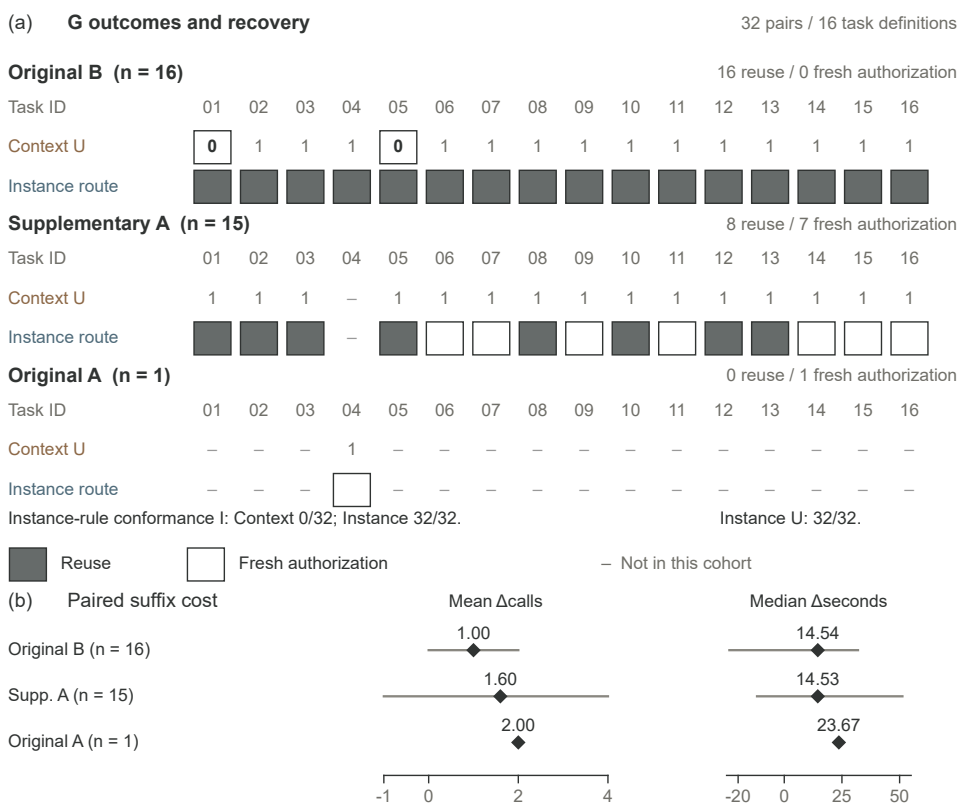


Fig. 4 All 32 delivered G pairs across 16 task definitions. (a) Task columns align across cohorts; dashes mean absent from that cohort's delivered pairs, not failure. Supplementary A omits 04; Original A includes only 04. I denotes instance-rule conformance, not safety under the context policy. Context has $I = 0$ throughout and $U = 1$ in 30 pairs; all bound arms end with $U = 1, I = 1$. Filled/open squares denote reuse/fresh authorization. (b) Diamonds show mean call and median elapsed-time differences for complete paired suffixes (Δ = bound minus context); segments show observed minima–maxima, not confidence intervals. Time includes decisions, tool interactions and terminal paths. Table 7 separates recovery calls

20/24 versus 24/24 at both caps, including four programmed context substitutions per cap. This 96-continuation study uses a later collection period and longer waits; its full results and the original failure decomposition are in Online Resource 2, A1–A2.

6.3 Two routes to accountable recovery

All 32 delivered G-bound mismatches recover. Original B's 16 continuations reuse the reviewed candidate; supplementary A has eight reuse and seven reauthorization paths; original A's single mismatch also uses reauthorization. Agents thus select different legal continuations under the same admission rule (Figure 4).

The observed recovery sequences are direct. After delivered feedback, each of the 24 reuse episodes makes one commit, and each of the eight fresh-authorization episodes

Table 7 Agent tool calls at delivered G handoffs. All entries except n are cohort means; post-commit calls occur after the first successful persistent recovery commit.

Collection	n	To recovery commit	After commit	Bound suffix	Context suffix	Δ
Original A	1	2.000	2.000	4.000	2.000	2.000
Original B	16	1.000	0.375	1.375	0.375	1.000
Supplementary A	15	1.467	1.533	3.000	1.400	1.600

Recovery comprises one commit in each of 24 reuse episodes and an authorization request followed by a commit in each of eight fresh-authorization episodes. The last column is bound minus context over complete suffixes, not calls beyond a route minimum. Values are recomputed from the archived event order and persistent transitions; no new runs are added.

requests authorization and then commits. No failed or exploratory tool call intervenes before these successful recovery commits. Later receipt or state queries are separate continuation actions (Table 7). The measurements cover agent continuation under the tested interface with immediate deterministic authorization.

An archived example makes the interface dependence explicit. In original-B pair `FORMAL-G-002-B`, the saved review response already supplies candidate `c1`, grant `a1` and authorized value 3412. The harness submits `c2` and returns `INSTANCE_MISMATCH`; the next agent action commits `c1` under `a1`, creating one transition. A later `get_receipt` call confirms that commit without another transition. Thus this episode contributes one recovery call and one subsequent confirmation call, versus zero agent calls in its context suffix. The saved feedback does not select a recovery route for the agent. Online Resource 1, S12.6 gives the event and state anchors.

Complete paired suffixes add a mean 1.00 tool call and median 14.54 seconds in original B, versus 1.60 calls and 14.53 seconds in supplementary A. The latter's seven reauthorization paths average 0.47 authorization requests across 15 pairs. Original A's single pair adds two calls and 23.67 seconds. Table 7 separates recovery, subsequent and context-side calls; Figure 4 preserves signed ranges. E/V/R delivered-episode results remain separate in Online Resource 1, S11–S12.

6.4 Exposure without rejection feedback

Agents shown a context-equivalent alternative reused the reviewed instance or obtained fresh authorization. The N family supplies such an alternative without forcing its submission. Across the collections, 186 arms were planned, 185 retain state and 127 reached review. Of these, 124 received a preview of a different-identity, same-context candidate in a saved model input, followed by a non-error response; these are arm-level exposures, and the two policy arms of a task share one prefix. Agents reused the reviewed instance in 123 cases and obtained matching authorization for the previewed candidate in one; there were zero unauthorized substitutions.

One original-B context arm supports only observation through its saved after-state, and its bound counterpart was not started; both integrity scores remain unknown. Excluding the partial arm leaves 126 reviewed arms, 123 exposures, 122 reuses and one

reauthorization (Online Resource 2, A7). N thus shows two continuity-preserving behaviors when an alternative is available. G measures recovery from delivered mismatch feedback; N contains no such feedback.

7 Discussion

7.1 Implications

Approval scope and continuation must be designed together. Context or content binding permits declared equivalences; instance binding turns a replacement into a rejection requiring agent action. The observed reuse, reauthorization and later confirmation show why a builder must specify both the allowed replacement and the interface for continuing after rejection.

These results suggest four choices for builders:

1. **Obtain the responsibility rule.** Have the workflow owner state whether approval covers a value, retained context, declared content or the reviewed artifact, and which replacements may reuse it. The measurements do not decide this organizational requirement.
2. **Cover the reusable content.** Put each decision-relevant field in the reuse key or check its approved value separately at execution; declare the comparator.
3. **Anchor binding and recovery.** Resolve identifiers through trusted lookup to immutable candidates; return the reviewed identifier or the state/evidence mismatch needed for reuse or fresh authorization. A valid original grant can support reuse even for an irreversible operation; stale state or evidence can require fresh authorization even for a reversible one.
4. **Check external objects at consumption.** Verify the reviewed digest and execute on the same captured bytes, as in the tested adapter.

The ToolGate adaptation illustrates this separation: framework callbacks express the obligations, while the application bridge supplies durable enforcement.

7.2 Threats to validity

Construct validity. Measured costs are agent-side tool calls and suffix time. Gate computation was not isolated; human re-inspection and approval-queue waiting were not measured. The deterministic reviewer immediately grants any valid request, whereas a deployed reauthorization request may wait, be refused or require transfer to an authorized reviewer. The observed routes and suffix times do not establish optimal route selection or production approval latency. Canonical equality is enforced at the gate; agreement between displayed and compared values was tested only in the browser fixture. The binding-scope study implements the selected comparators and a common executor, not the complete set of shared checks G, and normalizes money to two decimals instead of canonical JSON equality.

Internal validity. G substitutions are harness-created exposures (Section 6.1), and each cell has one realization. G recovery and N continuation were observed under the common preview–authorization instruction, with an additional reminder in N; no control without warnings isolates these prompts' effects. The completion comparisons

are not powered for a non-inferiority claim, and an interval that includes zero does not establish equivalence. The supplementary cohort and the output-budget study retain separate collection conditions and denominators. The deterministic protocol was amended after inspecting an initial run, and its 72 routes are crossed conditions over nine constructed cases, not independent samples.

External validity. Tasks are constructed, the reviewer is deterministic and the online host exercises a reduced single-field form of the contract. Natural mismatch prevalence, human approval cost and production benefit remain open empirical questions. The ToolGate comparison uses scripted decisions and an author-written commit bridge; it does not measure native default admission, autonomous agent performance or an efficiency advantage. The native-default study uses an in-process checkpointer and does not test restarts, concurrent replay or unauthorized post-approval mutation. Beyond the deterministic binding comparison, only two points of the design space are tested with agents; the remaining combinations have not been validated in a developer study.

Scope of the formal results. The formal and persistence results apply to the stated local observation model and atomic single-record successor (Section 3.3); multi-record or distributed enforcement requires additional coordination.

8 Conclusion

An approval's binding object determines which replacements it permits. The four-role correction contract connects this choice to authorized values, successor structure and field sources. Constructed comparisons, store realizations and a ToolGate adaptation make the obligations executable; paired identity-only handoffs reveal reuse and reauthorization, with confirmation costs accounted for separately. Together, they support implementing a declared approval scope at the write boundary with a corresponding recovery interface.

Statements and Declarations

Data and software availability

The study uses constructed task definitions and archived review checkpoints. The controlled experiment archive is fixed at commit 2645e5e18c900ea91c9c980e44195dc71e410432. The reference implementation, formal models and earlier historical evidence are retained at commit c6d512843c905cab6d8521dd8c914f7fb26d85ae. The prospective inputs and runner are fixed by a master manifest with SHA-256 7773563b40c8db30d6def121915f50fbad032537edde32afd4ba39ec3eea744b. The handoff archive at commit 475e66f549613aaaaa62c2ec81ffdee483636c54 retains the complete original formal ledger, raw responses, state snapshots, scorer, tables, figure sources and file hashes; Online Resource 1 maps the layers and reproduction steps. The archived supplementary deployment-A evaluation package preserves its separate freeze, complete attempt history, independent scores and source hashes; Online Resource 1, Section S12 identifies the offline analysis entry point. Reanalysis requires no new model calls.

New hosted runs require provider access. A reproduction guide provides the ordered offline workflow.

The deterministic native-approval and binding-scope experiments are supplied in `X1-X2-reproduction.zip`. The archive contains versioned sources, two saved executions, independent SQLite scoring and an offline verification entry point; Online Resource 2, A11 maps these records and their reproduction commands.

The manuscript source package includes the event-level recovery-cost readback, source hashes, an anchored recovery example and a script that recomputes Table 7 from the original and supplementary archives. The exact two-archive dependency list, hashes and offline command are in `REPRODUCE-RECOVERY.md`; `-output` keeps derived files separate. This decomposition adds no experimental runs and preserves the separate collection ledgers.

Ethics approval and consent to participate

The prospective experiment uses constructed tasks and a deterministic host reviewer, with no new human participants. In the historical annotation study, one author (X.W.) and one non-author volunteer independently coded benign completion outcomes; L.H. adjudicated their single inter-coder disagreement and separately reviewed eleven cases in which the strict trajectory label and the endpoint rule disagreed; a separate textual-acknowledgment audit with 39 human–rule disagreements was not adjudicated and is reported only as human–rule agreement. X.W. also completed the textual-acknowledgment annotation. Only pseudonymous annotator identifiers were collected. Both coders were informed of the task and data use, and the non-author coder participated voluntarily. The authors assessed the annotation of generated outputs as not requiring ethics approval. No animal subjects were involved.

CRediT authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing–original draft, and Writing–review & editing. Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing–review & editing. Chen Qile: Investigation and Validation. Peng Peng: Supervision and Writing–review & editing.

Funding and competing interests

This research received no external funding. The authors declare no competing interests.

AI-assisted preparation

DeepSeek assisted with data collection for the agent experiments. ChatGPT 6 Astra assisted with experimental verification and language polishing of selected manuscript passages. The candidate-record, authorization-tag and state-token glyphs in Fig. 1 were generated on 21 October 2026 with the built-in image-generation tool (the interface did not return the underlying image-model version); the supplied figure sources retain the

author-directed symbol specification, original asset and placement code; the candidate and authorization glyphs are also reused in Figs. 2 and 3. These diagrams were assembled in code with labels, values and paths set separately. The authors retain full responsibility for the research design, interpretation of the results, and the accuracy and integrity of the final manuscript.

Supplementary information

Online Resource 1 contains formal constructions and checks, full controlled workloads, historical records, prospective task and deployment definitions, scoring and interruption rules, complete outcome and recovery tables, and the artifact map. Online Resource 2 (Supplementary Addendum) reports later evidence: output truncation in deployment B (A1), an output-budget sensitivity study (A2), receipt semantics under uncertain commit outcomes (A3), local baseline probes (A4), the historical correction-chain audit protocol (A5), controlled-case summaries (A6), refreshed-preview exposure in the N family (A7), the ToolGate/public-reference comparison (A8), the extended specification and characterization (A9), the design-selection guidance (A10), and native-approval and binding-scope reproduction (A11). Both resources accompany this manuscript; their original and supplementary collection ledgers remain separate.

References

Arab BS, Gawlick D, Krishnaswamy V, et al (2018) Using reenactment to retroactively capture provenance for transactions. IEEE Transactions on Knowledge and Data Engineering 30(3):599–612. https://doi.org/10.1109/TKDE.2017.2769056

Barres V, Dong H, Ray S, et al (2026) τ^2-Bench: Evaluating conversational agents in a dual-control environment. In: Proceedings of the 43rd International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 306. PMLR, pp 6736–6782, URL https://proceedings.mlr.press/v306/barres26a.html

Bindschaedler L, Kossmann F, Liu C, et al (2026) Guarded commits: Transactional human approvals for LLM workflows. https://doi.org/10.48550/arXiv.2610.00037, URL https://arxiv.org/abs/2610.00037, preprint, version 1, arXiv:2610.00037

Buneman P, Khanna S, Tan WC (2001) Why and where: A characterization of data provenance. In: Database Theory—ICDT 2001, Lecture Notes in Computer Science, vol 1973. Springer, pp 316–330, https://doi.org/10.1007/3-540-44503-X_20

Buneman P, Chapman A, Cheney J (2006) Provenance management in curated databases. In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data, pp 539–550, https://doi.org/10.1145/1142473.1142534

Chen Z, Liu H, Xu D, et al (2026) Cordon: Semantic transactions for tool-using LLM agents. https://doi.org/10.48550/arXiv.2606.17573, URL https://arxiv.org/abs/2606.17573v1, arXiv preprint, version 1, arXiv:2606.17573

Cheney J, Chiticariu L, Tan WC (2009) Provenance in databases: Why, how, and where. Foundations and Trends in Databases 1(4):379–474. https://doi.org/10.1561/1900000006

Clark DD, Wilson DR (1987) A comparison of commercial and military computer security policies. In: 1987 IEEE Symposium on Security and Privacy, pp 184–194, https://doi.org/10.1109/SP.1987.10001

Gerrit Project (2026a) Gerrit code review: Review labels. URL https://gerrit-review.googlesource.com/Documentation/config-labels.html, official documentation, copyCondition and Sticky Votes. Accessed 21 October 2026

Gerrit Project (2026b) Gerrit code review: Submit requirements. URL https://gerrit-review.googlesource.com/Documentation/config-submit-requirements.html, official documentation. Accessed 21 October 2026

GitLab (2026) Merge request approval settings. URL https://docs.gitlab.com/user/project/merge_requests/approvals/settings/, official documentation. Accessed 21 October 2026

Gray J (1981) The transaction concept: Virtues and limitations. In: Proceedings of the Seventh International Conference on Very Large Data Bases, pp 144–154, URL https://www.vldb.org/conf/1981/P144.PDF

Haeberlen A, Kouznetsov P, Druschel P (2007) PeerReview: Practical accountability for distributed systems. In: Proceedings of the Twenty-First ACM SIGOPS Symposium on Operating Systems Principles, pp 175–188, https://doi.org/10.1145/1294261.1294279

Hardy N (1988) The confused deputy: (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review 22(4):36–38. https://doi.org/10.1145/54289.871709

He J, Yu D (2026) Beyond memory: A transactional continuity kernel for long-lived AI agents. https://doi.org/10.48550/arXiv.2608.11632, URL https://arxiv.org/abs/2608.11632v1, arXiv preprint, version 1, arXiv:2608.11632

He J, Veltri E, Santoro D, et al (2016) Interactive and deterministic data cleaning. In: Proceedings of the 2016 International Conference on Management of Data, pp 893–907, https://doi.org/10.1145/2882903.2915242

Jackson D (2002) Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11(2):256–290. https://doi.org/10.1145/505145.505149

Kumar AA (2026) Loopjacking: Hijacking human-in-the-loop approval. https://doi.org/10.48550/arXiv.2609.21081, URL https://arxiv.org/abs/2609.21081, preprint,

version 1, arXiv:2609.21081

Kung HT, Robinson JT (1981) On optimistic methods for concurrency control. ACM Transactions on Database Systems 6(2):213–226. https://doi.org/10.1145/319566.319567

LangChain (2026) Human-in-the-loop. URL https://docs.langchain.com/oss/python/langchain/human-in-the-loop, official documentation. Accessed 21 October 2026

Leucker M, Schallhart C (2009) A brief account of runtime verification. The Journal of Logic and Algebraic Programming 78(5):293–303. https://doi.org/10.1016/j.jlap.2008.08.004

Liu Y, Peng X, Cao J, et al (2026) ToolGate: Contract-grounded and verified tool execution for LLMs. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, pp 9653–9684, https://doi.org/10.18653/v1/2026.findings-acl.470, URL https://aclanthology.org/2026.findings-acl.470/

Lodderstedt T, Richer J, Campbell B (2023) OAuth 2.0 rich authorization requests. RFC 9396, RFC Editor, https://doi.org/10.17487/RFC9396, URL https://www.rfc-editor.org/rfc/rfc9396.html

Miller MS (2006) Robust composition: Towards a unified approach to access control and concurrency control. PhD thesis, Johns Hopkins University, Baltimore, Maryland, URL https://hdl.handle.net/1774.2/873

OpenTofu (2026) Command: apply. URL https://opentofu.org/docs/cli/commands/apply/, official documentation, Saved Plan Mode. Accessed 21 October 2026

Sabelfeld A, Myers AC (2003) Language-based information-flow security. IEEE Journal on Selected Areas in Communications 21(1):5–19. https://doi.org/10.1109/JSAC.2002.806121

Santos-Grueiro I (2026) Temporary authority, permanent effects: Commit-time authorization for LLM agents. https://doi.org/10.48550/arXiv.2607.10487, URL https://arxiv.org/abs/2607.10487v1, arXiv preprint, version 1, arXiv:2607.10487

Schneider FB (2000) Enforceable security policies. ACM Transactions on Information and System Security 3(1):30–50. https://doi.org/10.1145/353323.353382

Torlak E, Jackson D (2007) Kodkod: A relational model finder. In: Tools and Algorithms for the Construction and Analysis of Systems, Lecture Notes in Computer Science, vol 4424. Springer, pp 632–647, https://doi.org/10.1007/978-3-540-71209-1_49

Wang Y (2026) Approval laundering: Systematizing approval–execution binding failures in AI coding-agent harnesses. https://doi.org/10.48550/arXiv.2609.38983, URL

https://arxiv.org/abs/2609.38983, preprint, version 1, arXiv:2609.38983

World Wide Web Consortium (2013) PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, URL https://www.w3.org/TR/2013/REC-prov-dm-20130430/, edited by Luc Moreau and Paolo Missier. Recommendation dated 30 April 2013

Yakout M, Elmagarmid AK, Neville J, et al (2011) Guided data repair. Proceedings of the VLDB Endowment 4(5):279–289. https://doi.org/10.14778/1952376.1952378

Zhu J, Liu Z, Fan S, et al (2026) From approval to execution: Reconstruction-aware repair analysis for LLM-agent software. https://doi.org/10.48550/arXiv.2609.26529, URL https://arxiv.org/abs/2609.26529, preprint, version 1, arXiv:2609.26529